# Folhagem & Cia

Carlos Roma | Relatorio do desafio de back-end - PS CITi

## O que foi feito e como rodar

Implementei cadastro, consulta, edicao, exclusao, venda e valor do estoque, mantendo o CRUD de usuarios. Entraram os quatro bonus: busca por nome, ordenacao, esgotadas e seed. Stack: Node.js, Express, TypeScript, Prisma 5.22, PostgreSQL 16 e Docker.

Repositorio: https://github.com/carlosroma-dev/treinamento-crud-citi

Com Node.js 20+, Yarn e Docker Desktop aberto, rode no PowerShell:

```
git clone https://github.com/carlosroma-dev/treinamento-crud-citi.git
cd treinamento-crud-citi
Copy-Item .env.example .env
yarn install --frozen-lockfile
docker compose up -d --build
yarn migration
yarn seed
```

API: http://127.0.0.1:3001. Ajuste SERVER_PORT ou DATABASE_PORT no .env se necessario. O .env fica fora do Git. Codigo na main e em desafio/carlos-roma; comandos dos testes no README.

## Modelo, rotas e erros

Planta tem id inteiro automatico; nome e especie como textos nao vazios; preco inteiro positivo em centavos; estoque inteiro nao negativo; regaACadaDias inteiro a partir de 1; petFriendly booleano; e luminosidade igual a SOMBRA, MEIA_SOMBRA ou SOL_PLENO. createdAt e updatedAt sao datas automaticas. Os sete campos de negocio sao obrigatorios no cadastro; os inteiros respeitam Int32.

O cadastro recebe B: o exemplo abaixo sem id e datas. P representa a resposta completa. Dados ficticios:

```
{"id":1,"nome":"Planta A","especie":"Exemplo A","preco":8990,
 "estoque":2,"luminosidade":"MEIA_SOMBRA","petFriendly":false,
 "regaACadaDias":3,"createdAt":"2026-10-29T12:00:00.000Z",
 "updatedAt":"2026-10-29T12:00:00.000Z"}
```

Requisicoes e respostas usam JSON (Content-Type: application/json). :id e inteiro decimal positivo; [P] e uma lista de plantas completas. Edicao e venda tambem atualizam updatedAt. Na venda, quantidade deve ser inteira e positiva.

| Rota | Entrada ou exemplo | Sucesso e resposta | Erros |
|---|---|---|---|
| POST /plantas | Corpo B | 201: P | 400 |
| GET /plantas | Filtros opcionais abaixo; sem corpo | 200: [P] ou [] | 400 |
| GET /plantas/:id | /plantas/1; sem corpo | 200: P | 400, 404 |
| PATCH /plantas/:id | /plantas/1; {"preco":9500} | 200: P com preco 9500 | 400, 404 |
| DELETE /plantas/:id | /plantas/1; sem corpo | 204: sem corpo | 400, 404 |
| POST /plantas/:id/venda | /plantas/1/venda; {"quantidade":1} | 200: P com estoque 1 | 400, 404 |
| GET /plantas/estoque/valor-total | Sem corpo ou filtros | 200: total T abaixo | - |

| Rota | Entrada ou exemplo | Sucesso e resposta | Erros |
|---|---|---|---|
| GET /plantas/esgotadas | Mesmos filtros da lista; sem corpo | 200: [P com estoque 0] ou [] | 400 |

T, para preco/estoque 8990/2 e 2500/3: {"valorTotalEmCentavos":25480,"totalDeVasos":5}. Loja vazia: ambos zero.

Filtros combinaveis: luminosidade, petFriendly=true ou false, nome por trecho sem diferenciar maiusculas e ordem=preco ou nome, crescente. Exemplo: /plantas?luminosidade=MEIA_SOMBRA&petFriendly=false. Filtro valido sem resultado retorna 200 com [].

400 indica dado invalido ou estoque insuficiente; 404, planta inexistente; 500, falha inesperada em qualquer rota. Exemplo: {"message":"Planta nao encontrada."}. Campos extras, PATCH vazio e filtros repetidos tambem retornam 400.

## Como foi feito e conferido

Segui a ordem model/migration, repository, controller, rotas e testes. O controller valida e responde; so o repository consulta o Prisma. POST exige os sete campos; PATCH valida apenas os enviados e preserva os demais. Estoque 0 e petFriendly false sao validos.

A venda condiciona o decremento a estoque >= quantidade, na mesma escrita e em transacao com a leitura de retorno. A recusa nao altera o registro. O total usa BigInt e inteiros JSON exatos; acima do limite seguro de Number, o cliente precisa preservar a precisao na leitura.

Passaram 62 testes com PostgreSQL real (55 de plantas e 7 de usuarios), typecheck, build, subida com volume novo e 23 chamadas ao codigo compilado. Duas compras do ultimo vaso deram uma venda e uma recusa; dez compras para tres vasos deram tres vendas e sete recusas. Os testes tambem conferiram a preservacao dos dados em recusas e no PATCH.

A conta foi refeita por outro caminho: 8990 x 2 + 2500 x 3 = 25480 centavos e 5 vasos. Apos vender um vaso de 8990, restaram 16490 centavos e 4 vasos. O seed rodou duas vezes sem duplicar exemplos, e seu total foi conferido por SQL.

## Dificuldade e uso de IA

O volume do Docker tinha um Prisma Client que conhecia apenas User. Gerar o client no container e reiniciar resolveu. A inicializacao passou a gera-lo antes de subir a API. A revisao tambem preservou o parser JSON estrito dos usuarios.

Usei Codex, da OpenAI, amplamente no codigo, testes, revisoes e documento. Conferi cruzando requisitos, testes no banco e contas independentes. Ficou claro o cuidado com 0/false, PATCH parcial e venda atomica. Ainda preciso fazer minha leitura pessoal para explicar cada parte; os testes nao substituem isso.